Додаток 1. Лістинг коду програми

apps/lab1.cpp

```cpp
#include <iostream>
#include <FileSource.hpp>
#include <SymbolStore.hpp>
#include <Tokenizer.hpp>
#include <TokensView.hpp>

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cerr << "Usage: lab1 <filename>" << std::endl;

        return 1;
    }

    FileSource source(argv[1]);

    SymbolStore symbols;
    CharacterAttributes attributes;
    Tokenizer tokenizer(source, symbols, attributes);

    tokenizer.scan();

    TokensView view(std::cout);
    view.print(tokenizer.tokens(), symbols);

    for (const auto& error : tokenizer.errors()) {
        std::cout << std::format("Tokenizer | Error [{}:{}]: {}", error.row + 1, error.column + 1,
    }

    return 0;
}
```

src/CharacterAttributes/CharacterAttributes.hpp

```cpp
#pragma once

#include <array>
#include "Constants.hpp"

enum class Attribute {
    Whitespace,
    Digit,
    Letter,
    Delimiter,
    Comment,
    Invalid,
};

class CharacterAttributes {
private:
    std::array<Attribute, ASCII_TABLE_SIZE> attributes;

public:
    CharacterAttributes();

    [[nodiscard]] Attribute lookup(char character) const;
};
```

src/CharacterAttributes/CharacterAttributes.cpp

```cpp
#include "CharacterAttributes.hpp"

CharacterAttributes::CharacterAttributes() {
    for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
        attributes[i] = Attribute::Invalid;
    }

    // Whitespace
    attributes[' '] = Attribute::Whitespace;
    attributes['\t'] = Attribute::Whitespace;
    attributes['\n'] = Attribute::Whitespace;
    attributes['\v'] = Attribute::Whitespace;
    attributes['\f'] = Attribute::Whitespace;
    attributes['\r'] = Attribute::Whitespace;

    // Digits
    for (char ch = '0'; ch <= '9'; ch++) {
        attributes[ch] = Attribute::Digit;
    }

    // Letters (A-Z)
    for (char ch = 'A'; ch <= 'Z'; ch++) {
        attributes[ch] = Attribute::Letter;
    }

    // Delimiters
    attributes[';'] = Attribute::Delimiter;
    attributes['.'] = Attribute::Delimiter;
    attributes['='] = Attribute::Delimiter;
    attributes['\''] = Attribute::Delimiter;
    attributes[','] = Attribute::Delimiter;
    attributes[')'] = Attribute::Delimiter;
    attributes['$'] = Attribute::Delimiter;

    // Comment start
    attributes['('] = Attribute::Comment;
}

Attribute CharacterAttributes::lookup(char character) const {
    return attributes[static_cast<unsigned char>(character)];
}
```

src/Constants/Constants.hpp

```cpp
#pragma once

#include <cstddef>

constexpr size_t ASCII_TABLE_SIZE = 256;
```

src/Source/Source.hpp

```cpp
#pragma once

#include <SourcePosition.hpp>

class Source {
protected:
    SourcePosition position;

public:
    Source();
    Source(SourcePosition position);

    virtual ~Source() = default;

    virtual char current() = 0;
    virtual char read() = 0;
    virtual bool done() = 0;

    [[nodiscard]] unsigned int row() const;
    [[nodiscard]] unsigned int column() const;
};
```

src/Source/Source.cpp

```cpp
#include "Source.hpp"

Source::Source() = default;

Source::Source(SourcePosition position) : position(position) {}

unsigned int Source::row() const {
    return position.row();
}

unsigned int Source::column() const {
    return position.column();
}
```

src/Source/FileSource/FileSource.hpp

```cpp
#pragma once

#include <string>
#include <fstream>
#include "Source.hpp"

class FileSource : public Source {
private:
    std::ifstream file;
    char character;

public:
    FileSource(const std::string& filePath);

    char current() override;
    char read() override;
    bool done() override;
};
```

src/Source/FileSource/FileSource.cpp

```cpp
#include "FileSource.hpp"

FileSource::FileSource(const std::string& filePath) :
    file(filePath),
    character(static_cast<char>(file.get())) {}

char FileSource::current() {
    return character;
}

char FileSource::read() {
    position.advance(character);
    character = static_cast<char>(file.get());

    return character;
}

bool FileSource::done() {
    return file.eof();
}
```

src/Source/StringSource/StringSource.hpp

```hpp
#pragma once

#include <string>
#include "Source.hpp"

class StringSource : public Source {
private:
    std::string source;
    std::string::iterator iterator;

public:
    StringSource(std::string source);

    char current() override;
    char read() override;
    bool done() override;
};
```

src/Source/StringSource/StringSource.cpp

```cpp
#include "StringSource.hpp"

StringSource::StringSource(std::string source) :
    source(std::move(source)),
    iterator(this->source.begin()) {}

char StringSource::current() {
    return *iterator;
}

char StringSource::read() {
    position.advance(*iterator);
    iterator++;

    return *iterator;
}

bool StringSource::done() {
    return iterator == source.end();
}
```

src/SourcePosition/SourcePosition.hpp

```hpp
#pragma once

#include <cstdint>

class SourcePosition {
private:
    uint8_t tabSize = 4;

    unsigned int _row = 0;
    unsigned int _column = 0;

public:
    SourcePosition();
    SourcePosition(uint8_t tabSize);

    void advance(char character);
    [[nodiscard]] unsigned int row() const;
    [[nodiscard]] unsigned int column() const;
};
```

src/SourcePosition/SourcePosition.cpp

```cpp
#include "SourcePosition.hpp"

SourcePosition::SourcePosition() : SourcePosition(4) {}

SourcePosition::SourcePosition(uint8_t tabSize) :
    tabSize(tabSize) {};

void SourcePosition::advance(char character) {
    switch (character) {
        case '\n':
            _row++;
            _column = 0;
            break;

        case '\t':
            _column += tabSize;
            break;

        default:
            _column++;
            break;
    }
}

unsigned int SourcePosition::row() const {
    return _row;
}

unsigned int SourcePosition::column() const {
    return _column;
}
```

src/SymbolStore/SymbolStore.hpp

```hpp
#pragma once

#include <unordered_map>
#include <string>
#include <vector>
#include "Constants.hpp"

enum class SymbolType {
    Delimiter,
    Keyword,
    Literal,
    Identifier,
};

const size_t DELIMITERS_OFFSET = 0;
const size_t KEYWORDS_OFFSET = ASCII_TABLE_SIZE;
const size_t LITERALS_OFFSET = 500;
const size_t IDENTIFIERS_OFFSET = 1000;
const size_t MAX_TOKENS = 2000;

class SymbolStore {
private:
    std::array<std::string, MAX_TOKENS> symbols;
    std::unordered_map<std::string, size_t> keywords;
    std::unordered_map<std::string, size_t> identifiers;
    std::unordered_map<std::string, size_t> literals;

    void declareDelimiter(char character);
    void declareKeyword(const std::string& keyword);

public:
    SymbolStore();

    size_t resolveKeyword(const std::string& keyword);
    size_t resolveIdentifier(const std::string& identifier);
    size_t resolveLiteral(const std::string& literal);

    [[nodiscard]] bool isKeyword(const std::string& token) const;

    [[nodiscard]] const std::string& lookup(size_t code) const;
    [[nodiscard]] SymbolType lookupType(size_t code) const;
};
```

src/SymbolStore/SymbolStore.cpp

```cpp
#include "SymbolStore.hpp"

#include <format>
#include <stdexcept>

SymbolStore::SymbolStore() {
    // Initialize ASCII symbols
    for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
        declareDelimiter(static_cast<char>(i));
    }

    // Initialize keywords
    declareKeyword("PROGRAM");
    declareKeyword("CONST");
    declareKeyword("BEGIN");
    declareKeyword("END");
    declareKeyword("EXP");
}

void SymbolStore::declareDelimiter(const char character) {
    symbols[static_cast<unsigned char>(character)] = std::string(1, character);
}

void SymbolStore::declareKeyword(const std::string& keyword) {
    if (keywords.contains(keyword)) {
        throw std::invalid_argument(std::format("Keyword \'{}\' is already declared", keyword));
    }

    size_t code = KEYWORDS_OFFSET + keywords.size();

    if (code >= LITERALS_OFFSET) {
        throw std::overflow_error("Exceeded maximum number of keywords");
    }

    keywords[keyword] = code;
    symbols[code] = keyword;
}

size_t SymbolStore::resolveKeyword(const std::string& keyword) {
    if (!keywords.contains(keyword)) {
        throw std::invalid_argument(std::format("\'{}\' is not a keyword", keyword));
    }

    return keywords[keyword];
}

size_t SymbolStore::resolveIdentifier(const std::string& identifier) {
    if (!identifiers.contains(identifier)) {
        size_t code = IDENTIFIERS_OFFSET + identifiers.size();
        if (code >= MAX_TOKENS) {
            throw std::overflow_error("Exceeded maximum number of identifiers");
        }
        identifiers[identifier] = code;
        symbols[code] = identifier;
    }

    return identifiers[identifier];
}

size_t SymbolStore::resolveLiteral(const std::string& literal) {
    if (!literals.contains(literal)) {
        size_t code = LITERALS_OFFSET + literals.size();
```

```
        if (code >= IDENTIFIERS_OFFSET) {
            throw std::overflow_error("Exceeded maximum number of literals");
        }
        literals[literal] = code;
        symbols[code] = literal;
    }

    return literals[literal];
}

bool SymbolStore::isKeyword(const std::string& token) const {
    return keywords.contains(token);
}

SymbolType SymbolStore::lookupType(size_t code) const {
    if (code < KEYWORDS_OFFSET) {
        return SymbolType::Delimiter;
    }

    if (code < LITERALS_OFFSET) {
        return SymbolType::Keyword;
    }

    if (code < IDENTIFIERS_OFFSET) {
        return SymbolType::Literal;
    }

    if (code < MAX_TOKENS) {
        return SymbolType::Identifier;
    }

    throw std::out_of_range(std::format("Symbol code {} is out of range", code));
}

const std::string& SymbolStore::lookup(size_t code) const {
    if (code >= symbols.size()) {
        throw std::out_of_range(std::format("Symbol code {} is out of range", code));
    }

    return symbols[code];
}
```

src/Tokenizer/Tokenizer.hpp

```hpp
#pragma once

#include <vector>
#include <unordered_map>
#include "Source.hpp"
#include "SymbolStore.hpp"
#include "CharacterAttributes.hpp"

struct Token {
    size_t code;
    size_t row;
    size_t column;
};

struct Error {
    std::string message;
    size_t row;
    size_t column;
};

class Tokenizer {
private:
    Source& source;
    SymbolStore& symbols;
    CharacterAttributes& attributes;

    char character;
    std::string token;
    size_t code;

    std::vector<Token> _tokens;
    std::vector<Error> _errors;

    void scanWhitespaces();
    void scanInteger();
    void scanString();
    void scanComment();
    void scanDelimiter();
    void scanInvalid();

public:
    Tokenizer(Source& source, SymbolStore& symbols, CharacterAttributes& attributes);

    void scan();

    void addToken(const Token& token);
    void addError(const Error& error);

    [[nodiscard]] const std::vector<Token>& tokens() const;
    [[nodiscard]] const std::vector<Error>& errors() const;
};
```

src/Tokenizer/Tokenizer.cpp

```cpp
#include "Tokenizer.hpp"

#include <format>
#include "CharacterAttributes.hpp"

// Automata state: START
Tokenizer::Tokenizer(Source& source, SymbolStore& symbols, CharacterAttributes& attributes) :
    source(source),
    symbols(symbols),
    attributes(attributes),
    character(source.current()) {}

void Tokenizer::scan() {
    // Automata state: LOOP
    while (!source.done()) {
        switch (attributes.lookup(character)) {
            // Automata state: WHITESPACE
            case Attribute::Whitespace:
                scanWhitespaces();
                break;

            // Automata state: INTEGER
            case Attribute::Digit:
                scanInteger();
                break;

            // Automata state: STRING
            case Attribute::Letter:
                scanString();
                break;

            // Automata state: COMMENT_START
            case Attribute::Comment:
                scanComment();
                break;

            // Automata state: DELIMITER
            case Attribute::Delimiter:
                scanDelimiter();
                break;

            // Automata state: ERROR
            case Attribute::Invalid:
                scanInvalid();
                break;
        }
    }
    // Automata state: END
}

void Tokenizer::scanWhitespaces() {
    while (!source.done() && attributes.lookup(character) == Attribute::Whitespace) {
        character = source.read();
    }
}

void Tokenizer::scanInteger() {
    while (!source.done() && attributes.lookup(character) == Attribute::Digit) {
        token += character;
        character = source.read();
    };

```

```
63      // Automata state: WRITE
64      code = symbols.resolveLiteral(token);
65
66      addToken({
67          .code = code,
68          .row = source.row(),
69          .column = source.column() - token.length(), // Point to first character of the token
70      });
71
72      token.clear();
73  }
74
75  void Tokenizer::scanString() {
76      while (!source.done() && (attributes.lookup(character) == Attribute::Letter || attributes.loo
77          token += character;
78          character = source.read();
79      }
80
81      // Automata state: WRITE
82      if (symbols.isKeyword(token)) {
83          code = symbols.resolveKeyword(token);
84      } else {
85          code = symbols.resolveIdentifier(token);
86      }
87
88      addToken({
89          .code = code,
90          .row = source.row(),
91          .column = source.column() - token.length(), // Point to first character of the token
92      });
93
94      token.clear();
95  }
96
97  void Tokenizer::scanComment() {
98      size_t startRow = source.row();
99      size_t startCol = source.column();
100
101     character = source.read();
102
103     // Automata state: WRITE
104     if (character != '*') {
105         addToken({
106             .code = static_cast<size_t>('('),
107             .row = startRow,
108             .column = startCol,
109         });
110
111         return;
112     }
113
114     character = source.read();
115
116     // Automata state: COMMENT
117     while (!source.done()) {
118         // Automata state: COMMENT_CLOSE
119         if (character == '*') {
120             character = source.read();
121
122             // Automata state: COMMENT_END
123             if (character == ')') {
124                 character = source.read();
125
126                 return;
127             }
128         } else {
129             character = source.read();
```

```
130         }
131     }
132
133     // Automata state: ERROR
134     addError({
135         .message = "Comment not closed",
136         .row = startRow,
137         .column = startCol,
138     });
139 }
140
141 void Tokenizer::scanDelimiter() {
142     addToken({
143         .code = static_cast<size_t>(character),
144         .row = source.row(),
145         .column = source.column(),
146     });
147
148     character = source.read();
149 }
150
151 void Tokenizer::scanInvalid() {
152     addError({
153         .message = std::format("Invalid character \'{}\'", character),
154         .row = source.row(),
155         .column = source.column(),
156     });
157
158     character = source.read();
159 }
160
161 const std::vector<Token>& Tokenizer::tokens() const {
162     return _tokens;
163 }
164
165 const std::vector<Error>& Tokenizer::errors() const {
166     return _errors;
167 }
168
169 void Tokenizer::addToken(const Token& token) {
170     _tokens.push_back(token);
171 }
172
173 void Tokenizer::addError(const Error& error) {
174     _errors.push_back(error);
175 }
```

src/TokensView/TokensView.hpp

```hpp
#pragma once

#include <iostream>
#include <vector>
#include "Tokenizer.hpp"
#include "SymbolStore.hpp"

class TokensView {
private:
    std::ostream& out;

    static std::string typeLabel(SymbolType type);

public:
    TokensView(std::ostream& out);

    void print(const std::vector<Token>& tokens, const SymbolStore& symbols) const;
};
```

src/TokensView/TokensView.cpp

```cpp
#include "TokensView.hpp"

#include <format>

TokensView::TokensView(std::ostream& out) : out(out) {}

std::string TokensView::typeLabel(SymbolType type) {
    switch (type) {
        case SymbolType::Delimiter:
            return "Delimiter";
        case SymbolType::Keyword:
            return "Keyword";
        case SymbolType::Literal:
            return "Literal";
        case SymbolType::Identifier:
            return "Identifier";
    }
}

void TokensView::print(const std::vector<Token>& tokens, const SymbolStore& symbols) const {
    std::string border = "+-------+-----+-----+------------+---------+";

    std::string header = std::format(
        "| {:>5} | {:>3} | {:>3} | {:<10} | {:<7} |",
        "Code", "Row", "Col", "Type", "Value"
    );

    out << border << "\n";
    out << header << "\n";
    out << border << "\n";

    for (const auto& token : tokens) {
        auto value = symbols.lookup(token.code);
        auto type = typeLabel(symbols.lookupType(token.code));

        out << std::format(
            "| {:>5} | {:>3} | {:>3} | {:<10} | {:<7} |",
            token.code, token.row + 1, token.column + 1, type, value
        ) << "\n";
    }

    out << border << "\n";
}
```